

Avaliação: Projeto - Unidade III

1 Objetivo

Desenvolver um sistema para contar a quantidade de ocorrências de palavras em um texto usando uma tabela de dispersão (hash table) com tratamento de colisões utilizando listas encadeadas. A implementação deve ser realizada na linguagem C++.

2 Problema

Implemente um sistema para contar a quantidade de vezes que cada palavra aparece em um texto. O sistema deve permitir as seguintes operações:

1. **Adicionar Texto:** Adicionar um texto ao sistema para contagem.
2. **Contar Palavra:** Contar a quantidade de ocorrências de uma palavra específica no texto.
3. **Atualizar Palavra:** Atualizar o número de ocorrências de uma palavra específica.
4. **Remover Palavra:** Remover uma palavra do sistema.
5. **Verificar Palavra:** Verificar se uma palavra está presente no sistema e quantas vezes aparece.
6. **Imprimir Conteúdo:** Imprimir o conteúdo da tabela de dispersão para depuração e visualização.

3 Estrutura de Dados

A tabela de dispersão deve ser implementada com:

- Um array de listas encadeadas, onde cada posição do array contém uma lista de pares chave-valor.
- A chave será a palavra (string) e o valor será o número de ocorrências (inteiro).
- Uma função hash que distribua as palavras uniformemente entre as listas encadeadas.
- Tratamento de colisões utilizando listas encadeadas.

4 Implementação

A implementação deve ser realizada em C++. As operações devem ser implementadas como métodos de uma classe que representa a tabela de dispersão.

5 Especificação das Operações

5.1 Adicionar Texto

Descrição: Adiciona o texto ao sistema e conta a ocorrência de cada palavra.

Entrada: Texto (string).

Saída: Confirmação da operação (mensagem).

Exemplo de Entrada:

```
addText("o-gato-no-telhado");
```

Exemplo de Saída:

Texto adicionado. Contagem de palavras atualizada.

5.2 Contar Palavra

Descrição: Conta a quantidade de ocorrências de uma palavra específica no texto.

Entrada: Palavra (string).

Saída: Quantidade de ocorrências (inteiro) ou mensagem indicando que a palavra não foi encontrada.

Exemplo de Entrada:

```
int count = countWord("gato");
```

Exemplo de Saída:

A palavra "gato" ocorre 1 vez(es).

5.3 Atualizar Palavra

Descrição: Atualiza o número de ocorrências de uma palavra específica.

Entrada: Palavra (string), nova contagem (inteiro).

Saída: Confirmação da atualização (mensagem).

Exemplo de Entrada:

```
updateWord("gato", 2);
```

Exemplo de Saída:

Contagem da palavra "gato" atualizada para 2.

5.4 Remover Palavra

Descrição: Remove uma palavra do sistema.

Entrada: Palavra (string).

Saída: Confirmação da remoção (mensagem) ou mensagem indicando que a palavra não foi encontrada.

Exemplo de Entrada:

```
removeWord("gato");
```

Exemplo de Saída:

Palavra "gato" removida.

5.5 Verificar Palavra

Descrição: Verifica se uma palavra está presente no sistema e quantas vezes aparece.

Entrada: Palavra (string).

Saída: Mensagem indicando se a palavra está presente e sua contagem ou que a palavra não foi encontrada.

Exemplo de Entrada:

```
bool exists = checkWord("gato");
```

Exemplo de Saída:

A palavra "gato" está presente e ocorre 2 vez(es).

5.6 Imprimir Conteúdo

Descrição: Imprime todo o conteúdo da tabela de dispersão para depuração e visualização.

Entrada: Nenhuma.

Saída: Impressão do conteúdo de cada lista encadeada na tabela.

Exemplo de Entrada:

```
printTable();
```

Exemplo de Saída:

```
ndice    0: ("o", 1) -> ("gato", 2) -> NULL
ndice    1: ("telhado", 1) -> NULL
...
ndice   57: ("gato", 1) -> NULL
...
```

6 Função Hash

Para converter uma palavra em um índice para a tabela de dispersão, utilize a seguinte função hash:

```
#include <string>
```

```
const int TABLE_SIZE = 100; // Tamanho da tabela de dispersão
```

```
// Função hash que converte uma string para um índice
```

```
int hashFunction(const std::string& word) {
    unsigned long hash = 5381; // Valor inicial
    for (char c : word) {
        hash = ((hash << 5) + hash) + static_cast<unsigned long>(c); // hash * 33 + c
    }
    return hash % TABLE_SIZE; // Retorna o índice dentro dos limites da tabela
}
```

Descrição: A função hash converte uma string representando uma palavra em um índice para a tabela de dispersão. A função utiliza a soma dos valores ASCII dos caracteres da string e aplica um módulo com o tamanho da tabela para garantir que o índice esteja dentro dos limites.

Exemplo de Entrada e Saída:

```
#include <iostream>
```

```
int main() {
    std::string word = "gato";
    int index = hashFunction(word);
    std::cout << "Índice para palavra: " << word << " -> " << index << std::endl;
    return 0;
}
```

Exemplo de Saída:

```
Índice para palavra "gato": 57
```

7 Exemplos de Teste

7.1 Teste de Colisão

Para testar o tratamento de colisões, utilizamos a função hash fornecida para calcular índices para um conjunto de palavras e identificar palavras que colidem (geram o mesmo índice).

Função Hash:

```
#include <string>

const int TABLE_SIZE = 100; // Tamanho da tabela de dispersão

int hashFunction(const std::string& word) {
    unsigned long hash = 5381; // Valor inicial
    for (char c : word) {
        hash = ((hash << 5) + hash) + static_cast<unsigned long>(c); // hash * 33 + c
    }
    return hash % TABLE_SIZE; // Retorna o índice dentro dos limites da tabela
}
```

Palavras Testadas e Índices Gerados:

- "apple": Índice calculado = 42
- "orange": Índice calculado = 64
- "banana": Índice calculado = 76
- "grape": Índice calculado = 84
- "cherry": Índice calculado = 52
- "peach": Índice calculado = 42
- "mango": Índice calculado = 52
- "apricot": Índice calculado = 76
- "plum": Índice calculado = 64
- "date": Índice calculado = 84

Colisões Identificadas:

- Índice 42: "apple", "peach"
- Índice 52: "cherry", "mango"
- Índice 76: "banana", "apricot"
- Índice 64: "orange", "plum"
- Índice 84: "grape", "date"

Exemplo de Adição e Teste:

```
addText("apple - orange - banana - grape - cherry - peach - mango - apricot - plum - date");
```

Utilize a função `printTable()` para verificar a lista encadeada em cada índice e confirmar se as colisões foram corretamente tratadas.

Observação: Para garantir que a tabela de dispersão lida corretamente com colisões, adicione uma variedade de palavras e verifique os índices gerados. As listas encadeadas em cada índice devem conter todas as palavras que colidem e devem ser armazenadas e recuperadas conforme esperado.